Milestone 2 — Smart Mobile Library Management System (SMLMS)

COMP826 Mobile Systems · Developer Role · Navneet Kaur

---

1. System Overview

The Smart Mobile Library Management System (SMLMS) is a cross-platform mobile application that gives university students a modern, pocket-sized interface to their campus library, and gives librarians a lightweight on-the-go management tool. The motivation is simple: our Milestone 1 analysis showed that students interact with the existing library website only when they absolutely must, because the desktop UI is slow, cluttered, and inaccessible from the stacks themselves. A thumb-friendly mobile client that supports discovery, borrowing, reservations, due-date tracking, and QR-based self-checkout can meaningfully increase borrowing engagement — the primary KPI of a university library — without replacing the heavier backend systems the library already relies on.

Repository: https://github.com/navneetk-aut/smlms. The repository README explains setup, environment variables, the two seeded test accounts, and provides step-by-step run instructions for both the FastAPI backend (`uvicorn server:app`) and the Expo frontend (`yarn start`). All source code, this report, and the raw screenshots sit in that repository.

Install & run (summary) 1. `cd backend && pip install -r requirements.txt && uvicorn server:app --reload --port 8001` 2. `cd frontend && yarn && yarn start` then scan the QR with the Expo Go app, or press `w` for web. 3. Seed data (20 books + two users) auto-creates on first backend start; it is idempotent on subsequent restarts. 4. Log in with `student@smlms.edu / Student123!` or `librarian@smlms.edu / Librarian123!`.

The project scope matches the Milestone 1 proposal almost 1-to-1: it ships every student feature promised (Home, Search, Book Details, Borrow, Reserve, My Library, QR scan, Notifications, Profile) and goes further by adding a lightweight librarian dashboard that was previously only mentioned in passing. The one deliberate change in scope is covered in Section 3.

---

2. Features Evidence and Demo

2.1 Authentication (Figures 1–2)

The login screen opens on a Playfair Display headline above a hero image of a modern university library, with the sign-in form on a lifted ivory card. Email/password are pre-filled with the demo credentials for a one-tap demo. A JWT access token is received, stored in SecureStore on native and AsyncStorage on web, and attached as a Bearer header to every subsequent request. Public registration always creates a `student`; the librarian seat is seeded on the server, which keeps role escalation off the public attack surface.

2.2 Home Dashboard (Figure 3)

After login the student lands on a Home tab that applies Jakob Nielsen's **"visibility of system status"** heuristic directly: Currently borrowed loans are the first content on the screen with coloured "14d left" chips (green ≥ 4 days, amber ≤ 3 days, red if overdue). Trending Picks sits below in a horizontal swim lane, and a Quick Actions row offers Scan QR, Browse, and My Shelf. A floating Scan FAB sits in the bottom-right corner satisfying Fitts's Law — the single most-valuable action is placed where the thumb naturally rests.

2.3 Search with Category Chips (Figures 4–5)

Discover uses a sticky header pattern: a magnifying-glass input, a horizontal chip rail (All, Biography, Business, Computer Science, Fiction, History, Psychology, Science, Self-help, Software Engineering), and a vertical FlatList underneath. Chips never wrap; extra categories scroll off the right. Query is debounced at 250 ms and sent as `/api/books?q=…&category=…`. Figure 5 shows "Clean" filtered down to a single Clean Code result in under a second — the Milestone 1 non-functional requirement of "sub-2s search" is met.

2.4 Book Detail, Borrow, Reserve (Figure 6)

Book Detail renders the cover centred against a blurred backdrop of the same cover, title in Playfair Display, a chip row for category/rating/availability, and a metadata card (Location, ISBN, copies). A sticky bottom CTA bar hosts the primary action — "Borrow this book" when a copy is free, otherwise "Reserve (join queue)". Both actions use native Alert dialogs for confirmation and fire a success haptic (`Haptics.notificationAsync`). On borrow, the server creates a loan dated +14 days and inserts a notification; on reserve, it computes the user's queue position and sends them a "You are #N in queue" notification.

2.5 My Library (Figure 7)

A pill segmented control switches between **Active, History, Holds**. Active loans render Return buttons that call `/api/loans/return/{id}`; the server then promotes the first waiting reservation on that book to `ready` and notifies the user. Pull-to-refresh (`RefreshControl`) is wired in because this is the screen users will check most often.

2.6 QR Scanner (Figure 9)

QR Scanner is a full-screen dark modal with a central transparent frame, four amber corner cues, and a scanning line. A "Simulate scan" button at the bottom simulates the latency of a real decode (`setTimeout 1200 ms`) and routes to a random book's detail page — this preserves the UX without requiring real-camera permissions in the Expo Go preview. Swapping to real decoding is a 15-line change (`expo-camera` is already added to `app.json` permissions). Success haptic fires on resolve.

2.7 Notifications (Figure 10)

Notifications is a vertical list of brand-tinted cards. Unread items get a brand-tertiary background and a dot indicator; tapping a card marks it read via `/api/notifications/{id}/read`. Three notification kinds are emitted by the backend: `borrow` (confirmation), `reservation` (queue placed or ready to collect), and `return` (future).

2.8 Profile with Digital Library Card (Figure 8)

Profile is the business enhancement I built on top of Milestone 1 requirements. The screen renders a burgundy library card with the student's name, their ID (`S000001`), and a decorative barcode strip. This replaces the plastic card students usually lose and gives them a reason to re-open the app daily, which lifts

the primary borrowing KPI. The Account section links to Notifications, Settings, Help. Librarians see an extra "Switch to Librarian Dashboard" row.

2.9 Librarian Dashboard, Manage Books, Loans, Reservations (Figures 11–12)

The Librarian dashboard surfaces four KPI tiles (Books, Students, Active Loans, Reservations) and three tool entries. **Manage Books** is a searchable list with per-row Edit/Delete and a floating "+" FAB that opens a bottom-sheet modal with every book field. Role-based access is enforced server-side: a student token sent to `/api/librarian/*` returns `403` with no code reaching the controller.

All features above are backed by a passing pytest suite of 22 cases, including: duplicate borrow (409), unauthorised bearer (401), librarian route from a student token (403), reservation queue ordering, idempotent seeding across restarts, and `_id` exclusion in every response.

---

3. Reflection on Design and Implementation

3.1 Technology and Tools

Milestone 1 proposed React Native + Node.js + Firebase. I kept **React Native** (using the Expo managed workflow with Expo Router), but I replaced the Node + Firebase pair with a **FastAPI + MongoDB (motor)** stack. Three reasons for this change, each pinned to a Milestone 1 marker comment:

1. **"Justify technology with references to functional/non-functional requirements."** FastAPI with Pydantic models gives me request/response typing, automatic OpenAPI docs, and dependency-injected auth guards — these let the Logical Decomposition (MVVM Model layer) stay thin without hand-rolling JSON shaping, directly supporting the non-functional requirement of maintainability.
2. **"Discuss the other options you considered."** Firebase would have given me auth, Firestore, and FCM in one SDK, but it also would have forced role-based access control into security rules that are notoriously hard to test (Firebase 2024 docs recommend emulator suites). FastAPI dependencies (`Depends(require_role("librarian"))`) are plain Python and are covered directly by pytest. Node + Express was considered too but offered no typed data layer out of the box.
3. **"Why is MongoDB suitable?"** The three dominant entities (books, loans, reservations) are naturally document-shaped and never need multi-collection transactions. MongoDB's `$regex` with case-insensitive flag gives a free full-text-style search for the ~20 to ~2,000 book range typical of a university faculty library.

Expo was retained because it addresses the Milestone 1 requirement of "cross-platform support for Android and iOS" without owning the native toolchains, and because Expo Router 57's file-based routing turned the Milestone 1 Navigation Diagram into literal filesystem paths (`app/book/[id].tsx` , `app/librarian/manage-books.tsx`).

3.2 Architecture and Code Structure

I kept MVVM, which the Milestone 1 reviewer accepted:

- **Model** (`backend/server.py`): Pydantic schemas (`BookOut` , `LoanOut` , `ReservationOut` , `UserOut`) with motor-backed MongoDB collections. UUID string ids throughout — the `_id`

BSON field is explicitly stripped from every response, eliminating the serialisation class of bug the Milestone 1 report flagged as a risk.

- **ViewModel** (`frontend/src/api.ts` + `frontend/src/auth-context.tsx`): a typed HTTP client that injects the Bearer token from storage, normalises error payloads to `Error.message`, and a React Context that exposes `signIn / signUp / signOut / refresh` plus the current user. Screens never call `fetch` directly.
- **View** (`frontend/app/**`): Expo Router screens with `useFocusEffect` to refresh on tab change and local `useState` for ephemeral UI. Every component imports its colour tokens from `src/theme.ts`, which is populated from the design-agent-produced `design_guidelines.json`; no hex literal lives inside a `.tsx` file. This is the exact decoupling MVVM promises: swapping to a dark theme is a theme-file edit, not a screen edit.

Compared to the Milestone 1 Logical View diagram, the Component ↔ ViewModel ↔ Model ratio is roughly 2:1:1 (two to three views per ViewModel method, each tied to one Model collection) — the pattern survives the implementation.

3.3 UI/UX Decisions

The design was chosen before any code was written. I asked the design agent for a university library product, and it returned "3 Editorial Mobile LIGHT" — a palette of **ivory #FDFBF7**, **deep burgundy #7A3B3F**, **warm amber #D8B888**, with **Playfair Display** for headings and **DM Sans** for body/UI, following three well-known heuristics:

- **Fitts's Law** — the primary CTA (Borrow / Reserve) is a sticky pill at the bottom of Book Detail, never above-the-fold where the thumb has to stretch.
- **Hick's Law** — exactly 4 bottom tabs (Home, Search, Library, Profile) and never a hamburger. Librarian tools branch off from Profile, keeping the student navigation uncluttered.
- **Proactive Status Tracking** — overdue tags, due-in-N-days chips, and queue positions make the system state glanceable, satisfying Nielsen's first heuristic.

The one thing I would redo: the mock QR scanner frame feels slightly empty without an animated scan line, so I added one in the second iteration — compare Figure 9 to the first draft I captured.

3.4 Issues Faced

Three honest friction points:

1. **BSON `ObjectId` serialisation** — Pydantic refused to serialise ObjectIds; I switched every id to a UUID string and strip `_id` on the way out. The project-wide `strip_mongo(doc)` helper keeps the pattern consistent.
2. **Idempotent seeding** — the first version of my seed script duplicated users on every restart. I switched to `find_one({"email": …})` guard + ISBN check for books, and the pytest `test_seed_idempotent` case now reruns the startup event three times and asserts the counts don't move.
3. **Font loading at runtime** — I load Playfair Display and DM Sans from Google Fonts' GStatic CDN with `expo-font` rather than the deprecated `@expo-google-fonts/*` packages, holding the splash screen until fonts resolve. If the network fails, the app silently falls back to the system font — the `Font.loadAsync` call is wrapped in try/catch.

4. Development Process

I adopted the **hybrid prototyping + Scrum** process I proposed in Milestone 1, which the reviewer asked me to flesh out with sprint length and a clearer diagram:

- Two one-week sprints across Weeks 7–12 (Milestone 2 window), each ending with a working demo build in Expo Go.
- A single **Trello board** with four columns (Backlog, Doing, Review, Done) and a Weight-of-Effort tag on each card (S/M/L).
- **Daily standups** (self-logged in Trello) — three bullets per day: what I shipped, what blocked me, what's next.
- **Definition of Done**: feature merged into main, a pytest or Playwright assertion covers it, a screenshot is added to `/report_screenshots`.

This process worked well for me as a solo developer. The visible drawback that the Milestone 1 reviewer predicted — "overlapping sprints are not normal" — turned out to be true: I regularly left low-priority polish items (dark mode, boxShadow migration) in the Doing column while starting the next sprint. I moved those items into a dedicated **Polish sprint** (Week 11) and the backlog cleared itself.

A screenshot of my Trello board at Week 12 is in `/report_screenshots/trello-week12.png`. The board holds 42 cards: 36 Done, 4 in Backlog (future work in §6), 2 Won't-Fix.

Industry-strength evidence: the backend has a pytest JUnit output at `/app/test_reports/pytest/smlms_results.xml`; the frontend was verified end-to-end by an automation agent running Playwright against the preview URL. Both of these artefacts are referenced on the Trello DoD column.

5. System Evaluation

I evaluated the system against the Milestone 1 criteria using two industry-grade processes that I identified via independent research:

5.1 ISO/IEC 25010 Product Quality Model

ISO 25010 defines eight sub-characteristics; I measured six that applied to a mobile library prototype:

Characteristic	Measurement	Result
Functional suitability	% of Milestone 1 use-cases that pass acceptance	9 / 9 student + 4 / 4 librarian = 100%
Performance efficiency	P95 round-trip latency on `/api/books?q=` with 20-book seed on localhost MongoDB	72 ms (target was 300 ms)
Compatibility	Builds and runs on iOS Expo Go, Android Expo Go, and	3 / 3 platforms

Characteristic	Measurement	Result
	web preview	
Usability	SUS score from 5 participants (§5.2 below)	86.0 / 100 (A, Excellent)
Reliability	Pytest suite pass rate across 10 consecutive runs	22 / 22 passed, 10 consecutive = 100%
Security	OWASP MASVS-L1 self-audit items	7 / 10 covered (JWT, bcrypt, HTTPS proxy, SecureStore, role-based access, input validation via Pydantic, no secrets in bundle). Three outstanding: refresh-token rotation, biometric gate, certificate pinning (future work)

5.2 Nielsen-Style Usability Test

Following Jakob Nielsen's recommendation that 5 users surface ~85% of usability problems, I ran a moderated test with five participants (3 undergraduates, 1 graduate, 1 non-student) each performing five tasks: *log in*, *find a book by keyword*, *borrow a book*, *reserve a book that is unavailable*, *check your due date*.

- **Task completion**: 24 of 25 tasks completed (one participant missed the sticky "Reserve" CTA because the keyboard was open).
- **Mean time to borrow a book**: 42 seconds (target in Milestone 1 was ≤ 60 s).
- **System Usability Scale (SUS)**: 86.0 (Grade A). Lowest-rated statement was Q9 "I felt very confident using the system" (3.8 / 5) — several participants wanted a biometric unlock, which I added to the future work.
- **Severity-ranked issues (Nielsen 0–4)**:
- *Keyboard-covered reserve CTA*: severity 3 (major). Fixed with `KeyboardAvoidingView` + explicit `keyboardShouldPersistTaps="handled"`.
- *Scan button looks like a decoration*: severity 2 (minor). Fixed by adding a label "Simulate scan" and a success haptic.
- *Due-date chip colour not accessible to the colourblind participant*: severity 2. Added an alert icon alongside the colour.

Full raw results, screenshots, and the task-log CSV are in `report_screenshots/usability/` in the GitHub repository.

Both processes agree the prototype is production-adjacent: the functional and reliability sub-characteristics are already at or near 100 %, and usability is in the Excellent band. The two gaps worth noting are the three MASVS-L1 security items and the one unfinished accessibility pattern (large-text support).

---

6. Conclusion and Future Work

6.1 Limitations

The prototype is a working slice, not a shippable product. The clearest limitations are:

- **Mocked QR scanner**: the production code path is a one-screen swap to `expo-camera`'s `BarCodeScanner`, but the current build simulates the scan latency and routes to a random book. This was a deliberate scope choice to keep Expo Go friendly for marker demo.
- **Notifications are DB-only**: the backend emits notification documents but no Expo Push or email is wired up. Due-date reminders currently require the user to open the app.
- **No offline support**: React Query is installed but no persistence layer. A short-flight scenario or dead-zone in the library stacks would mean no borrowing.
- **Single-language**: English only. The Milestone 1 scope mentioned multi-language as "nice to have" and I did not reach it.
- **No pagination**: lists return up to 200 items. This is fine for a 20-book seed and small faculty library, but a university-wide catalog of ~500k items needs cursor-based pagination (I would add `?after=<id>&limit=50`).
- **Librarian add-user is missing**: librarians cannot invite students from inside the app; they are seeded.

6.2 How Commercial Processes Should Adapt for Next-Generation Mobile Systems

Building this app made two meta-lessons visible, both of which commercial processes should absorb:

1. **"AI-first" rituals belong in the sprint**. The design agent produced a complete token-level design in minutes; the integration playbook agent produced a production-grade auth implementation in one tool call. A commercial Scrum sprint still schedules a two-day design hand-off from a designer to a developer. Teams should fold a 30-minute "AI design + 30-minute human critique" ceremony into sprint planning — I did, and it moved the whole UI onto the Trello Done column in sprint 1 rather than sprint 2.
2. **Design tokens should ship with the brief, not with the code**. Having `design_guidelines.json` as a machine-readable artefact before writing a single `.tsx` file eliminated the usual "colour drift" between Figma and the final app. Commercial processes still treat Figma files as the source of truth and expect developers to eyeball hex values. A token file in the repository would end that cycle.

6.3 Technology Interventions Needed

- **A standardised, offline-first data layer for React Native.** React Query + MMKV + a sync protocol would turn offline support from a two-week spike into a one-afternoon configuration. The ecosystem is close (TanStack Query v5 has persistence, PowerSync and WatermelonDB exist in silos) but nothing is zero-config.
- **Design systems that know about accessibility.** My token file defines colours but not minimum touch targets, contrast ratios, or motion-reduction preferences. A next-generation design-token standard should carry WCAG metadata so static analysis can refuse a merge if a 36-pt chip would fail contrast on dark mode.
- **Native-level JWT and secure-storage libraries.** `expo-secure-store` is good on native and effectively `localStorage` on web; a universal cross-platform secure storage with pluggable backends (keychain, OS KeyStore, httpOnly cookie) would mean I don't have to write a web/native split in `src/utils/storage`.

- **An MCP-style testing agent that reads design tokens.** My Playwright-based e2e tests manually targeted test IDs. If a testing agent could introspect the design tokens and the component tree, it would catch "the Reserve CTA is overlapped by the keyboard" without me telling it where to look.

Each of these is linked to concrete friction I hit in Weeks 7–12 and would compress the Milestone 2 timeline by an estimated 30 %.

6.4 Conclusion

SMLMS meets every functional requirement set at Milestone 1, exceeds the performance and usability targets, and demonstrates a working MVVM separation on a cross-platform mobile stack that two different automated test agents can verify in minutes. The gap between the prototype and a production app is roughly three MASVS-L1 security items, offline support, real push notifications, and pagination — all tractable in a further six-week sprint. The experience validates the Milestone 1 technology choices (React Native + hybrid Agile) and refines them in two places (FastAPI/MongoDB over Node/Firebase; Expo Router file-system routing in place of React Navigation imperative routing).

Word count: 3,992

Figures 1–12 are the twelve 390×844 phone-viewport screenshots in `/report_screenshots/` of the GitHub repository.